# Begin

Nicholas Alan Gilbert Emperical Exercise #2 10-26-26

## Table of Contents

Set Up

Data Handling

Task 1 - Compute Log Returns

Task 2 - Annaulized Returns

Task 3 - Data Visualization

# Set Up

We will import necessary libraries and custom helper-functions written in another file.

In [1]:
```
%load_ext autoreload
%autoreload 2

import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as st
import numpy as np
import ipywidgets

#Custom Helper Functions
import helper.tools as tools
```

In [3]:
```
stocks = tools.StockSearch()
```

```
VBox(children=(HBox(children=(Combobox(value='', description='Ticker:',
options=('MMM', 'AOS', 'ABT', 'ABBV', …
```

In [13]:
```
stocks.start
```

Out[13]:
```
datetime.date(2000, 1, 1)
```

In [14]:
```
stocks.end
```

Out[14]:
```
datetime.date(2026, 10, 26)
```

In [18]:
```
stocks.tickers = ['XLC', 'XLY', 'XLP', 'XLE', 'XLF', 'XLV', 'XLI', 'XL

stocks.tickers
```

Out[18]:
```
['XLC', 'XLY', 'XLP', 'XLE', 'XLF', 'XLV', 'XLI', 'XLB', 'XLRE', 'XL
K', 'XLU']
```

In [19]:
```
df = stocks.data.copy()
```

```
[**********100%**********]  11 of 11 completed
```

In [20]:
```
df
```

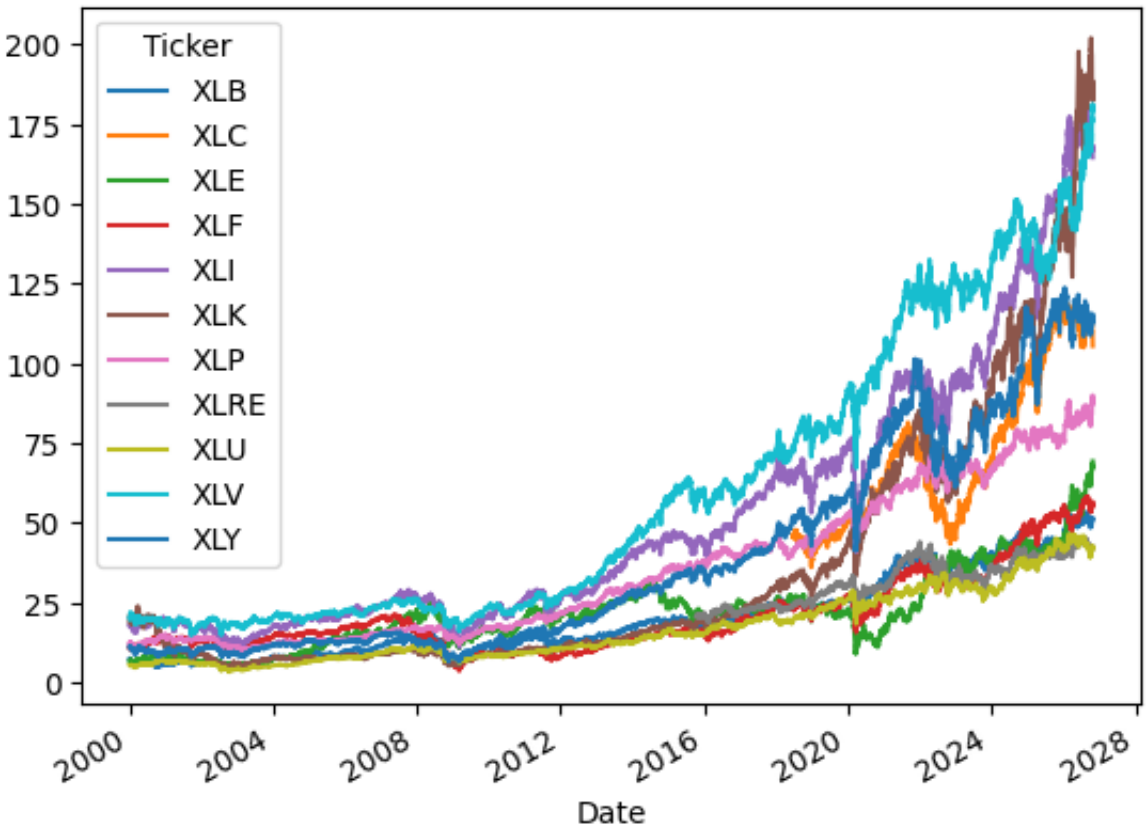
Out[20]:

| Price | | | | | | |
|---|---|---|---|---|---|---|
| **Ticker** | **XLB** | **XLC** | **XLE** | **XLF** | **XLI** | **XLK** |
| **Date** | | | | | | |
| **2000-01-03** | 7.168345 | NaN | 6.569214 | 11.013208 | 17.982185 | 20.577808 |
| **2000-01-04** | 7.099332 | NaN | 6.445558 | 10.531755 | 17.485386 | 19.533834 |
| **2000-01-05** | 7.392622 | NaN | 6.615584 | 10.449008 | 17.407461 | 19.243847 |
| **2000-01-06** | 7.582393 | NaN | 6.870625 | 10.907898 | 17.641241 | 18.605864 |
| **2000-01-07** | 7.578085 | NaN | 6.944041 | 11.088442 | 18.313396 | 18.930651 |
| **...** | ... | ... | ... | ... | ... | ... |
| **2026-10-19** | 50.950001 | 110.089996 | 67.800003 | 55.060001 | 166.679993 | 185.960007 |
| **2026-10-20** | 51.230000 | 110.190002 | 67.970001 | 55.720001 | 166.800003 | 185.440002 |
| **2026-10-21** | 51.320000 | 108.970001 | 68.160004 | 56.040001 | 168.309998 | 188.539993 |
| **2026-10-22** | 51.689999 | 107.419998 | 67.570000 | 56.650002 | 166.729996 | 182.339996 |
| **2026-10-23** | 51.310001 | 105.510002 | 69.330002 | 55.900002 | 164.619995 | 183.229996 |

6743 rows × 22 columns

In [21]:
```
df['Close'].plot()
```

Out[21]: <Axes: xlabel='Date'>




# Data Handling

Back to Top

## Log Returns

Log returns are computed under the assumption of *continous time*. It comes from the inverse of the exponential function $e^t$

$\ln(e^t) = t$

Read: the natural-log of $e$ *to the something* is that something.

### Context

Where We Definied Returns Before;

$ r_t = $ return at time period t

$ p_t = $ asset price at time t

$ t = $ time-period eg. daily, weekly, monthly

Asset Return:

$$ r_t = \frac{p_t}{p_{t-1}} - 1$$

Then a series of successive returns of $n$ periods compounded is represented geometrically:

$$r_{\text{cumulative}} := (\prod_{i=1}^{n} 1 + r_i) - 1 = [(1+r_1) \cdot (1+r_2) \cdot (1+r_3) \dots (1+r_n)] - 1$$

This is a more expensive operation, adding 1s to each entry then performing $n$ multiplications.

# Natural Log to the Rescue

Some nice arithmetic properties of using ln() returns is that compounding is additive.

$$\ln\left( \frac{p_t}{p_{t-1}} \right) = \ln( p_t) - \ln( p_{t-1})$$

$$\ln\left( {p_t}\cdot{p_{t-1}} \right) = \ln( p_t) + \ln( p_{t-1})$$

## We Define Log-Returns:

$$r_t := \ln(p_t) - \ln(p_{t-1})$$

$$r_t := \ln(p_{\text{today} } ) - \ln(p_{\text{yesterday} })$$

# Calculation Method

We can apply NumPy's `ln()` function to an entire array of asset prices, then use pandas' `.shift()` method to subract a whole shifted array from the other.

Will look something like this:

```
log_returns = np.log(data['Close']) -
np.log(data['Close']).shift(1)
```

# Missing Data

At this point we will 'produce' a NaN value in our return calculation from our first date, we don't have a previous data point to compute a return from that point. So we'll just drop it. Other stock's will exhbit NaN's, say they weren't listed on an exchange at that point, so no data occurs for them.

## Interpretation

Working with log-returns is nice mathematically. But it will differ from the true return an investor will see if they have their money invested. For example, between 2018-02-26 and 2026-08-28.

### Example

#### AAPL's Average Daily Log-Return: 0.096%

#### AAPL's Average Daily Real Return: 0.123%

In [ ]:

# Task 1

# Compute Log-Returns

In [22]:
```
log_returns = np.log(df['Close']) - np.log(df['Close']).shift(1)
log_returns
```

Out[22]:

| Ticker | XLB | XLC | XLE | XLF | XLI | XLK | |
|---|---|---|---|---|---|---|---|
| Date | | | | | | | |
| 2000-01-03 | NaN | NaN | NaN | NaN | NaN | NaN | |
| 2000-01-04 | -0.009674 | NaN | -0.019003 | -0.044700 | -0.028016 | -0.052065 | -0.0 |
| 2000-01-05 | 0.040482 | NaN | 0.026037 | -0.007888 | -0.004467 | -0.014957 | 0. |
| 2000-01-06 | 0.025346 | NaN | 0.037827 | 0.042980 | 0.013340 | -0.033715 | 0. |
| 2000-01-07 | -0.000568 | NaN | 0.010629 | 0.016416 | 0.037393 | 0.017306 | 0. |
| ... | ... | ... | ... | ... | ... | ... | |
| 2026-10-19 | 0.000589 | -0.002087 | -0.006469 | -0.008320 | -0.006459 | 0.013480 | -0.0 |
| 2026-10-20 | 0.005481 | 0.000908 | 0.002504 | 0.011916 | 0.000720 | -0.002800 | 0. |
| 2026-10-21 | 0.001755 | -0.011134 | 0.002791 | 0.005727 | 0.009012 | 0.016579 | -0.0 |
| 2026-10-22 | 0.007184 | -0.014326 | -0.008694 | 0.010826 | -0.009432 | -0.033437 | 0. |
| 2026-10-23 | -0.007379 | -0.017941 | 0.025714 | -0.013328 | -0.012736 | 0.004869 | -0. |

6743 rows × 11 columns

In [23]:
```
log_returns.isna().sum(), log_returns[log_returns.index > '2018'].isna
```

Out[23]:
```
(Ticker
 XLB        1
 XLC     4645
 XLE        1
 XLF        1
 XLI        1
 XLK        1
 XLP        1
 XLRE    3967
 XLU        1
 XLV        1
 XLY        1
 dtype: int64,
 Ticker
 XLB       0
 XLC     117
 XLE       0
 XLF       0
 XLI       0
 XLK       0
 XLP       0
 XLRE      0
 XLU       0
 XLV       0
 XLY       0
 dtype: int64)
```

## Note on NaN Handling

So far we can just ignore the NaNs. They aren't 'in the way' for calculating log-returns at this point. The 'sanity check' above tells us there isn't large holes in the data. I know that ticker TRMD wasn't listed until 2018 compared to our other tickers. So we are missing data for historical reasons. Plus we will have NaN from calculating returns.

If we know some caclulations will choose to 'ignore NaNs' or 'include NaNs', this is what looking up the code documentation is for.

In [24]:
```
sum_stats = tools.summary_statistics_table(log_returns)

sum_stats
```

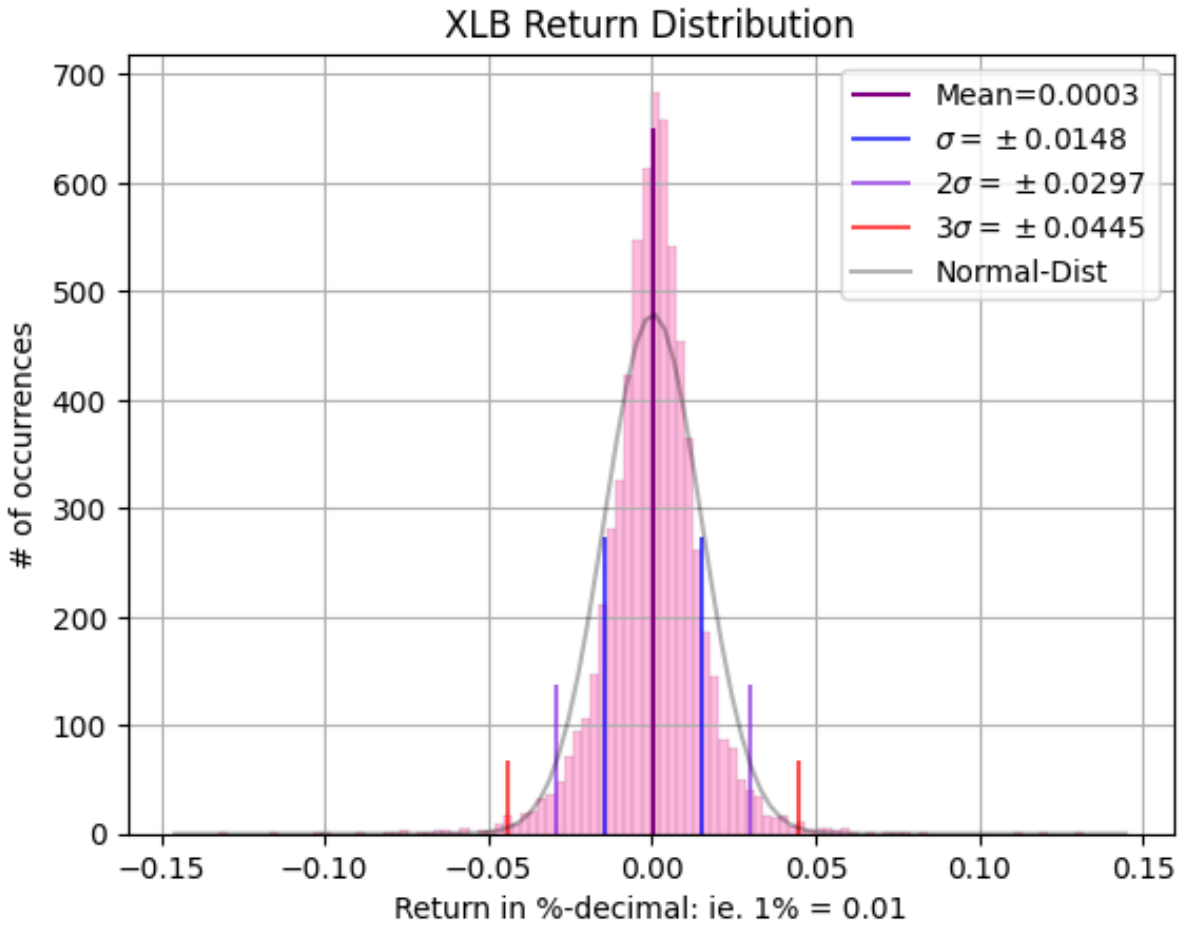
Out[24]:

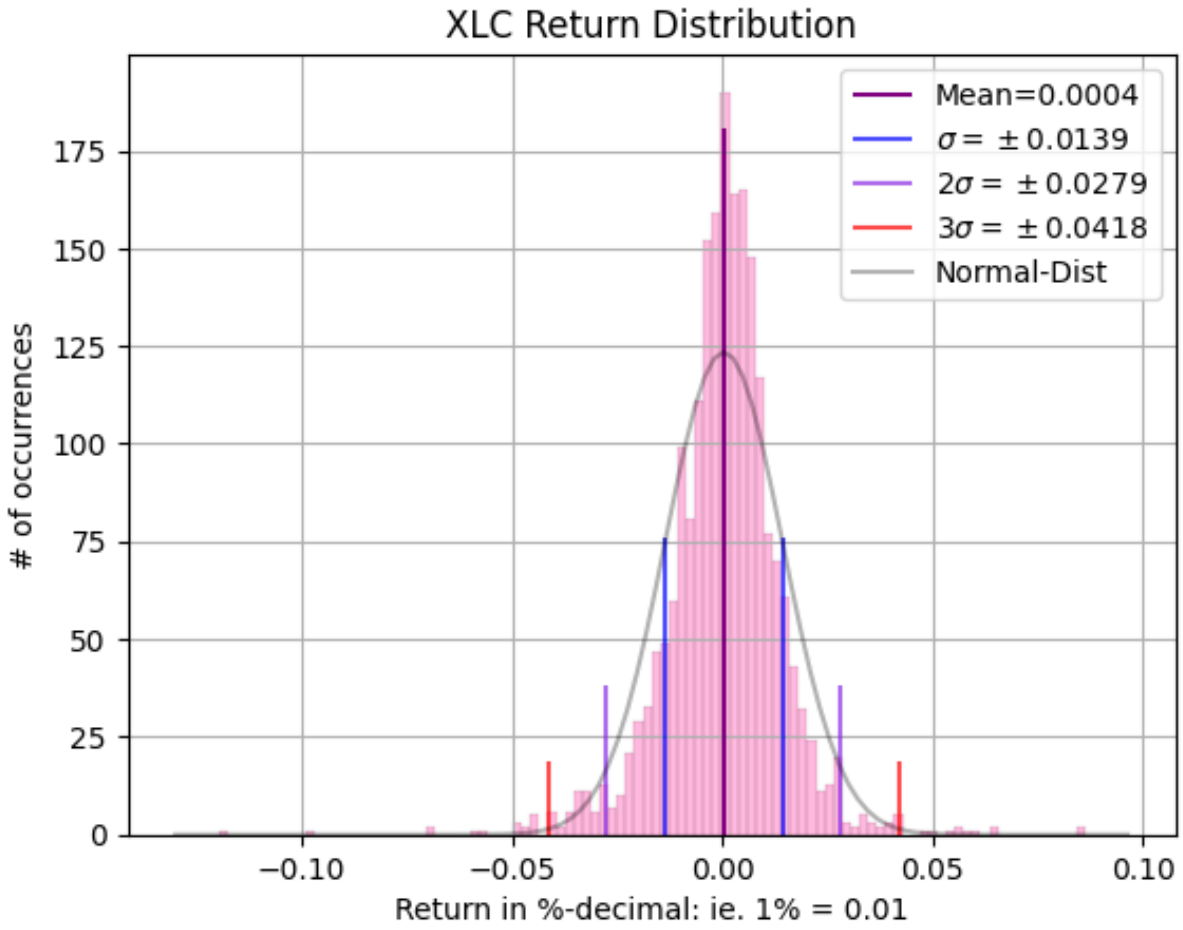
| | XLB | XLC | XLE | XLF | XLI | XLK |
|---|---|---|---|---|---|---|
| **Mean** | 0.000292 | 0.000395 | 0.000350 | 0.000241 | 0.000328 | 0.000324 |
| **Median** | 0.000745 | 0.001067 | 0.000821 | 0.000616 | 0.000813 | 0.001063 |
| **Standard-Deviation** | 0.014826 | 0.013948 | 0.018165 | 0.017757 | 0.013335 | 0.016288 |
| **Skewness** | -0.211880 | -0.456690 | -0.649857 | -0.159880 | -0.332843 | 0.080895 |
| **Kurtosis** | 6.730035 | 7.106145 | 12.215804 | 15.469194 | 7.993847 | 7.074342 |
| **Min** | -0.132527 | -0.119679 | -0.224910 | -0.182322 | -0.120408 | -0.148662 |
| **Max** | 0.131534 | 0.086087 | 0.152504 | 0.152394 | 0.119126 | 0.149296 |

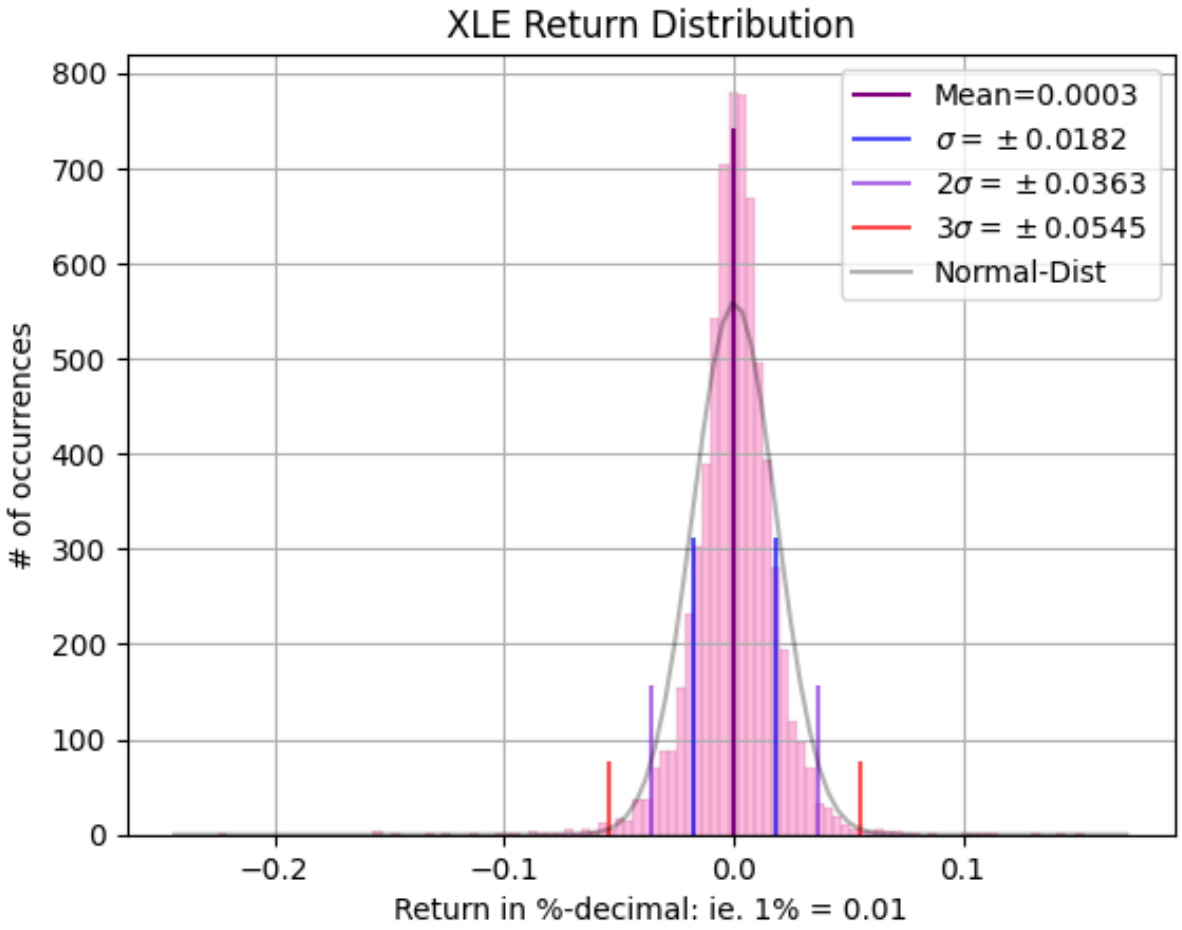
In [25]:
```
np.exp(sum_stats.loc['Mean']) - 1
```

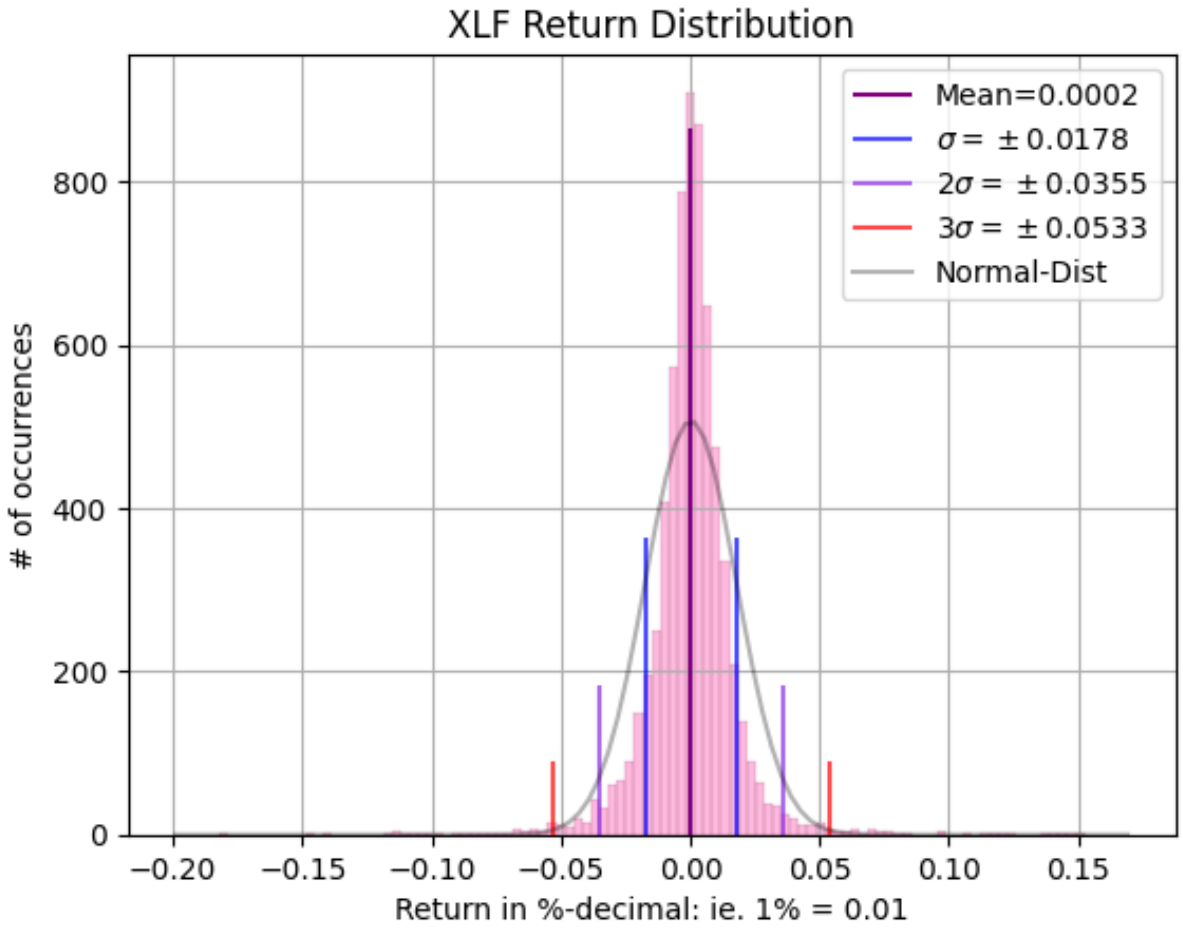
Out[25]:
```
XLB     0.000292
XLC     0.000395
XLE     0.000350
XLF     0.000241
XLI     0.000328
XLK     0.000324
XLP     0.000296
XLRE    0.000260
XLU     0.000305
XLV     0.000323
XLY     0.000344
Name: Mean, dtype: float64
```

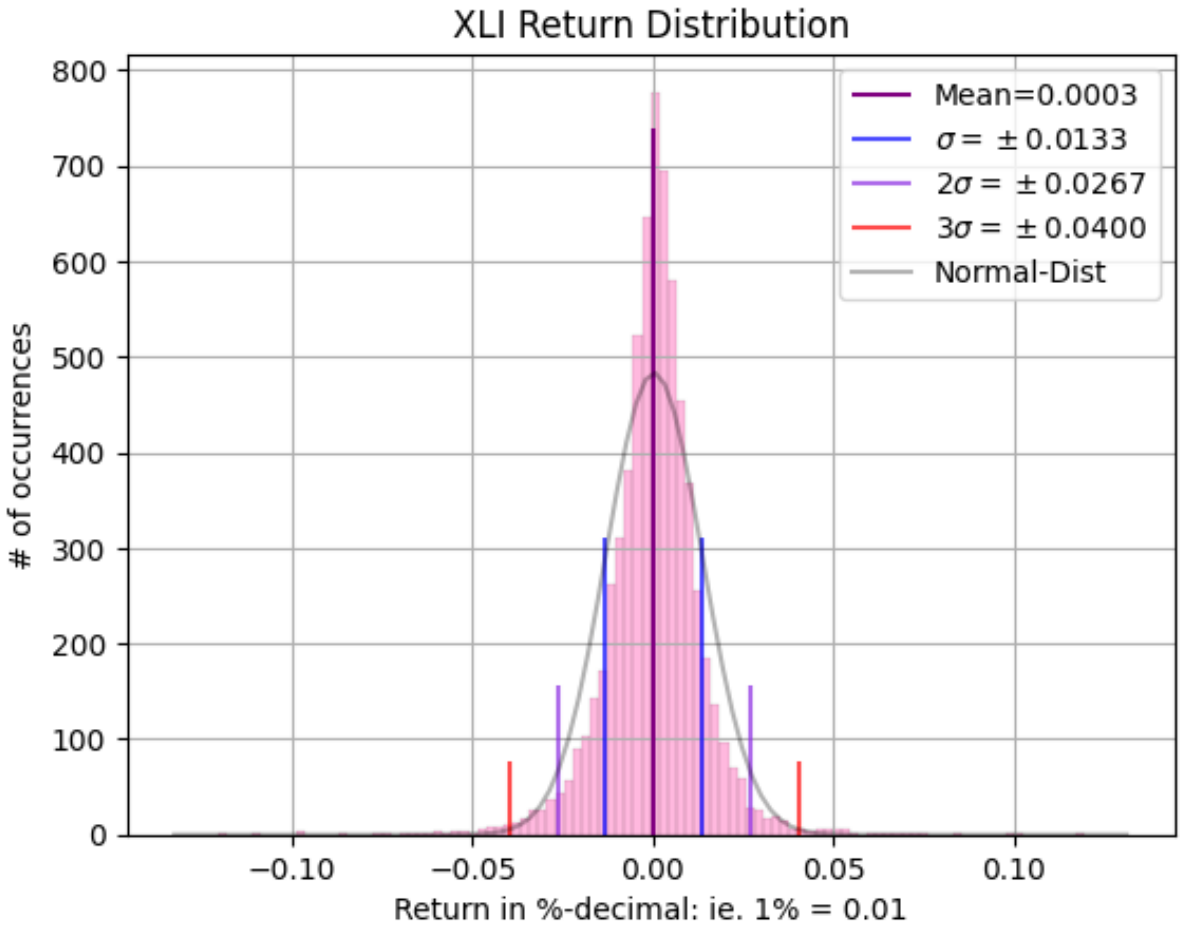
In [27]:
```
tools.return_histograms(log_returns)
```

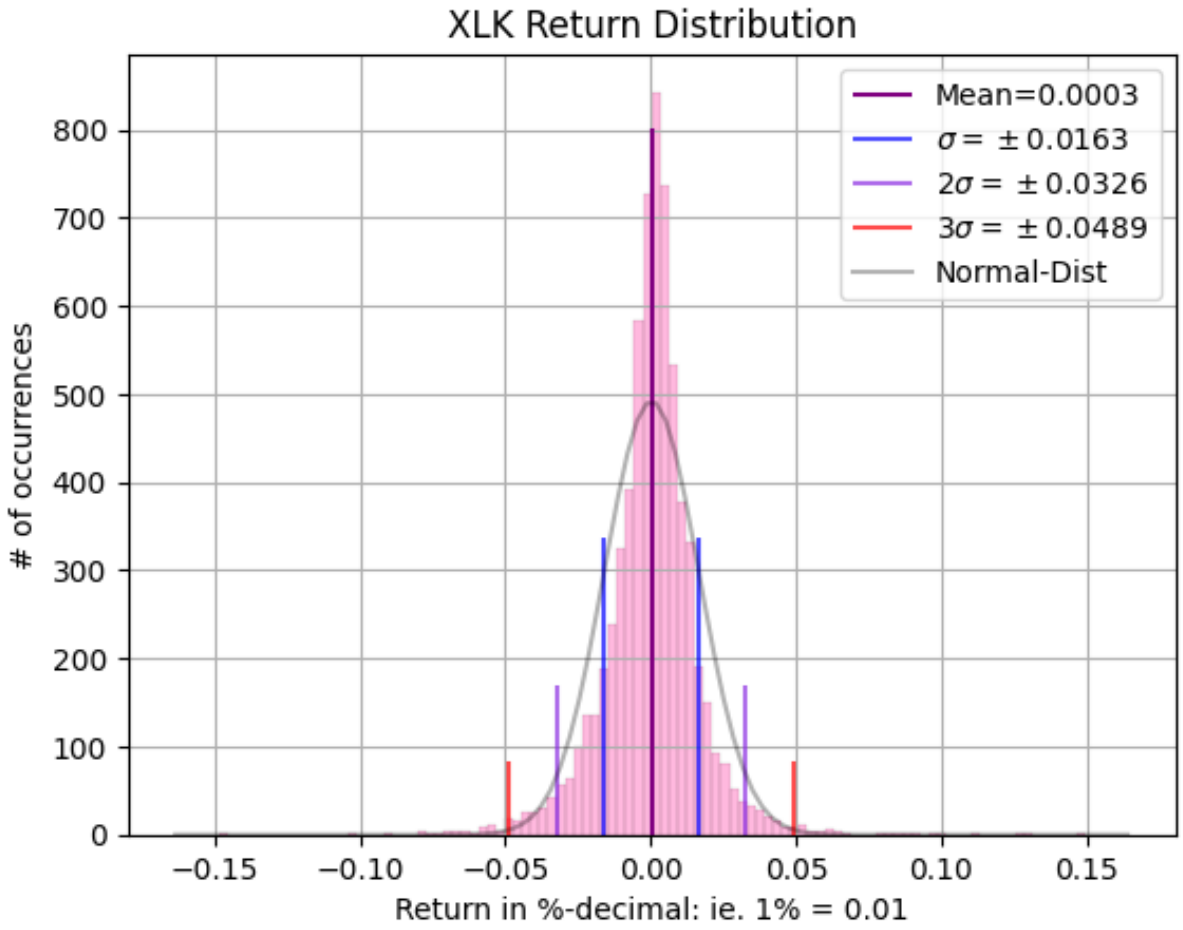

XLB Return Distribution
Mean=0.0003
σ = ±0.0148
2σ = ±0.0297
3σ = ±0.0445
Normal-Dist
# of occurrences
0
100
200
300
400
500
600
700
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15
Return in %-decimal: ie. 1% = 0.01

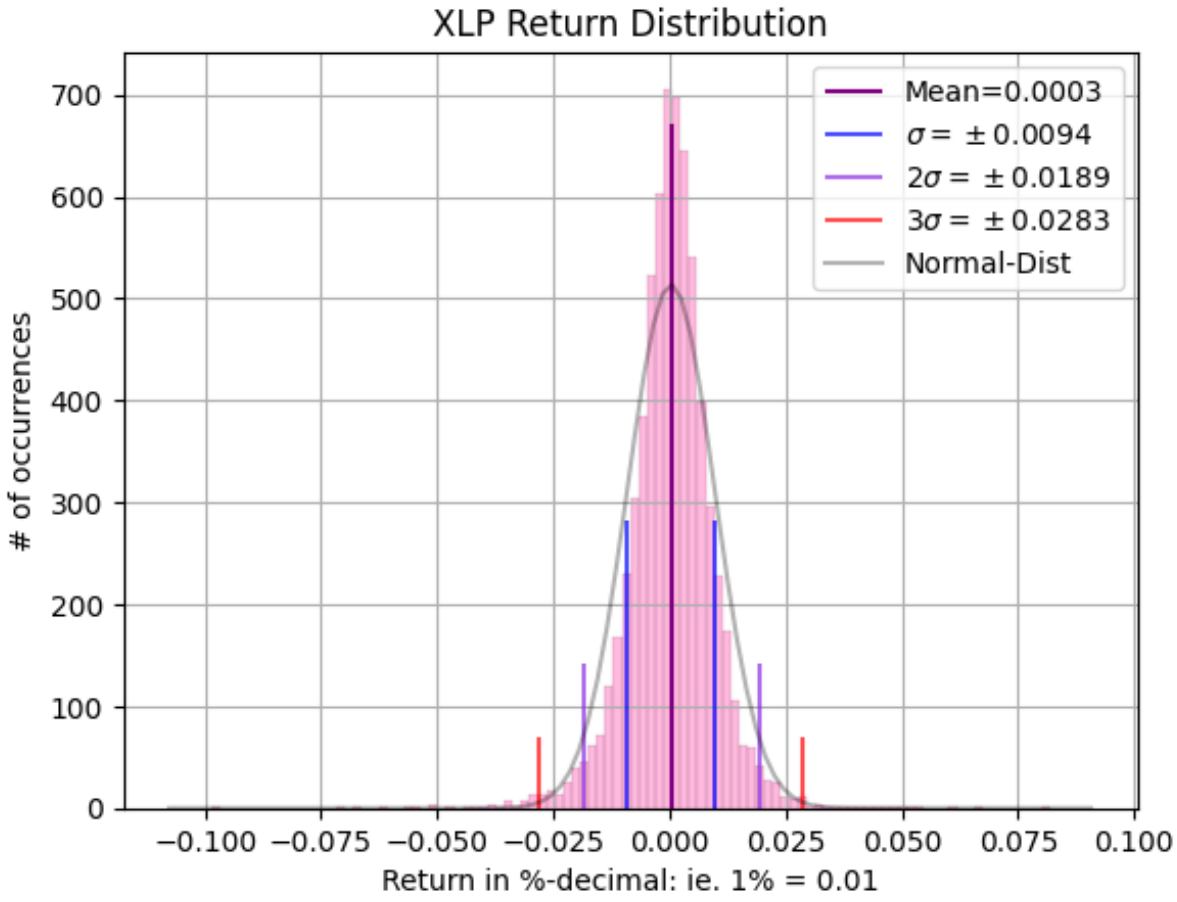

XLC Return Distribution
Mean=0.0004
σ = ±0.0139
2σ = ±0.0279
3σ = ±0.0418
Normal-Dist
# of occurrences
0
25
50
75
100
125
150
175
−0.10
−0.05
0.00
0.05
0.10
Return in %-decimal: ie. 1% = 0.01

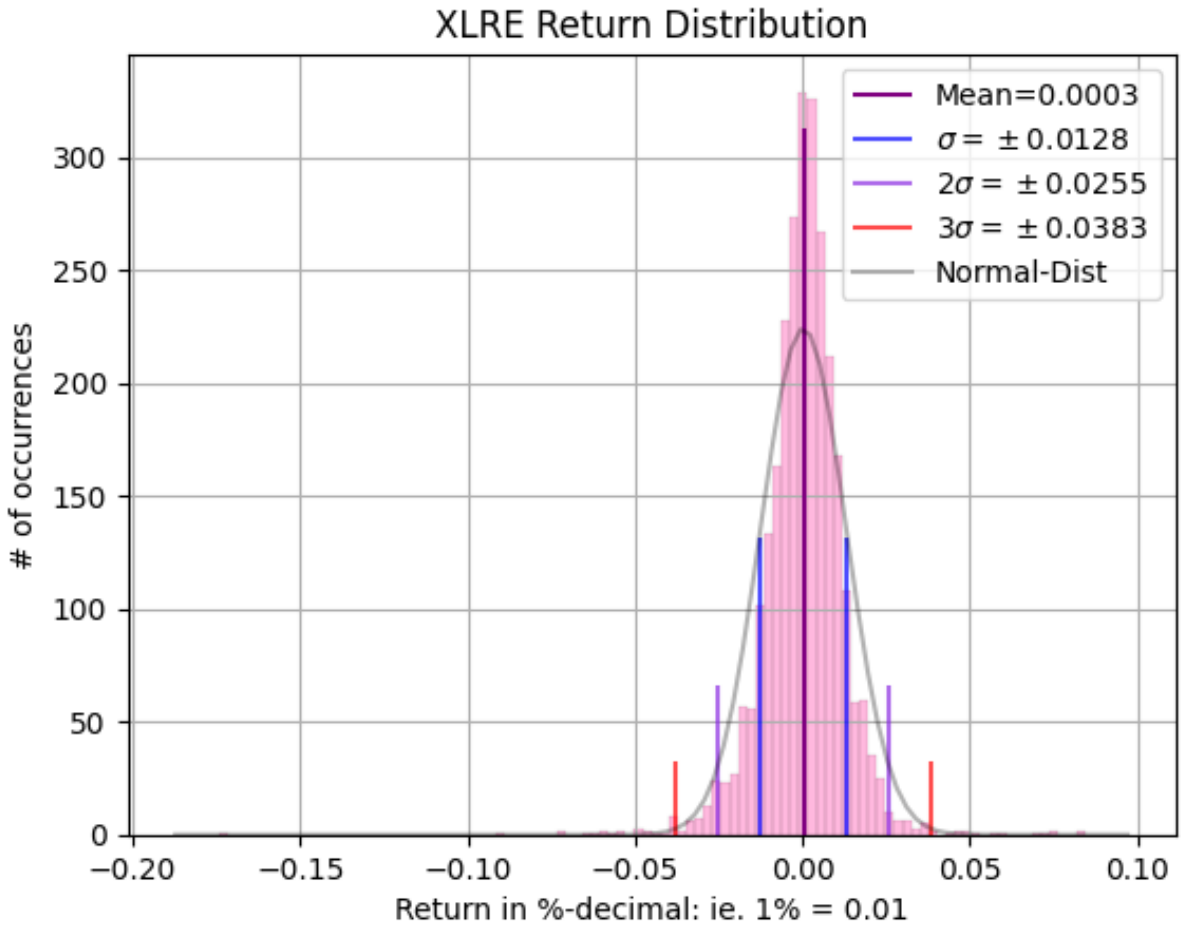

XLE Return Distribution
Mean=0.0003
$\sigma = \pm 0.0182$
$2\sigma = \pm 0.0363$
$3\sigma = \pm 0.0545$
Normal-Dist
# of occurrences
800
700
600
500
400
300
200
100
0
−0.2
−0.1
0.0
0.1
Return in %-decimal: ie. 1% = 0.01

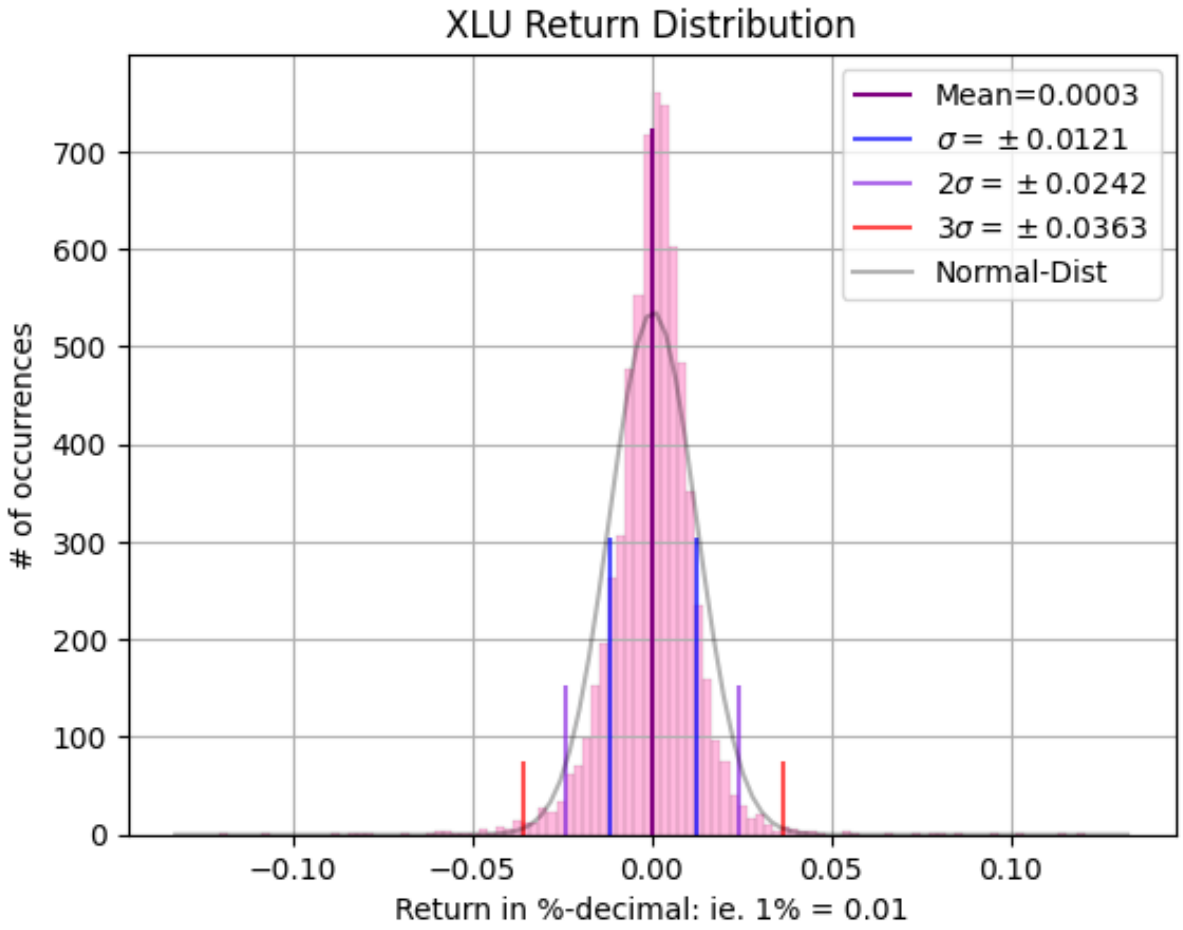

XLF Return Distribution
Mean=0.0002
$\sigma = \pm 0.0178$
$2\sigma = \pm 0.0355$
$3\sigma = \pm 0.0533$
Normal-Dist
# of occurrences
0
200
400
600
800
−0.20
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15
Return in %-decimal: ie. 1% = 0.01

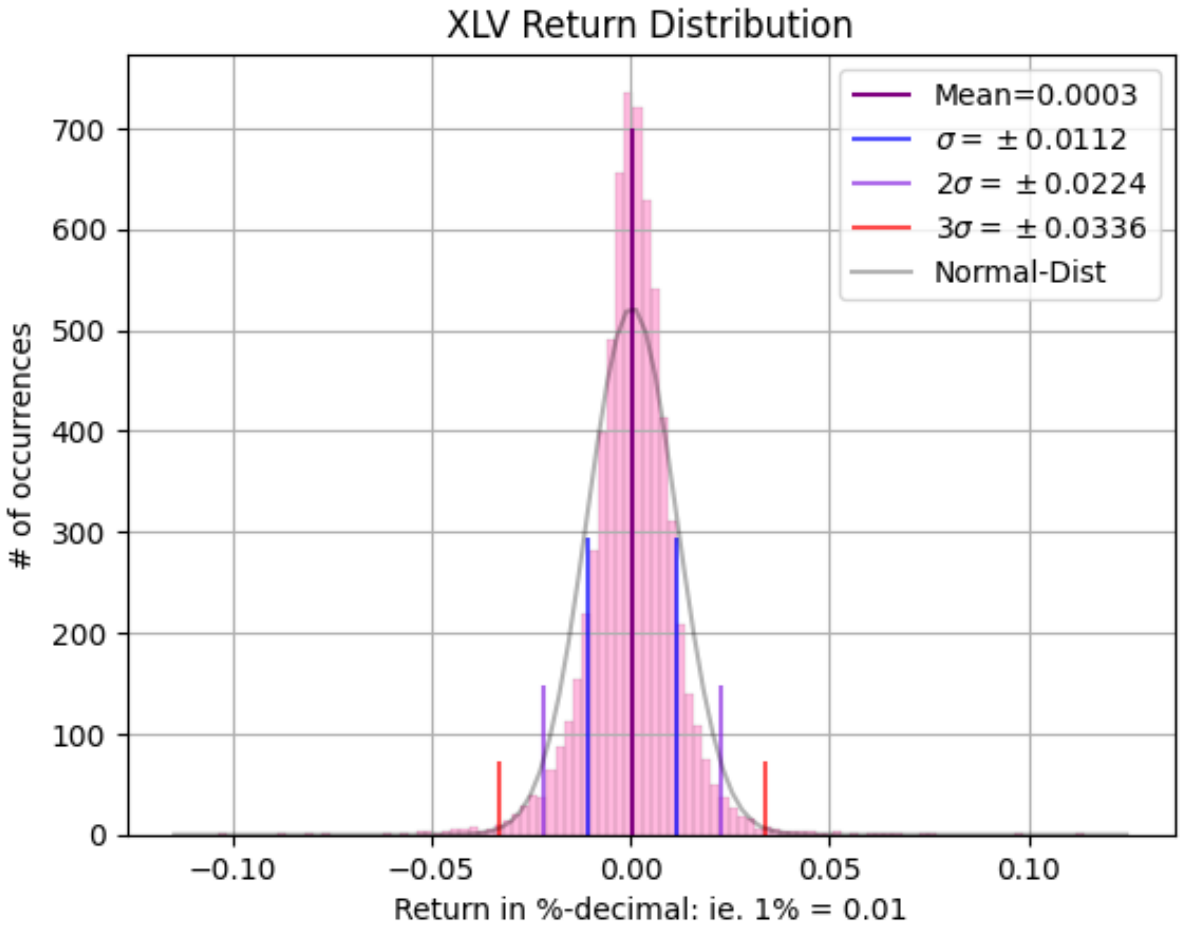

XLI Return Distribution
Mean=0.0003
$\sigma = \pm 0.0133$
$2\sigma = \pm 0.0267$
$3\sigma = \pm 0.0400$
Normal-Dist
# of occurrences
0
100
200
300
400
500
600
700
800
−0.10
−0.05
0.00
0.05
0.10
Return in %-decimal: ie. 1% = 0.01

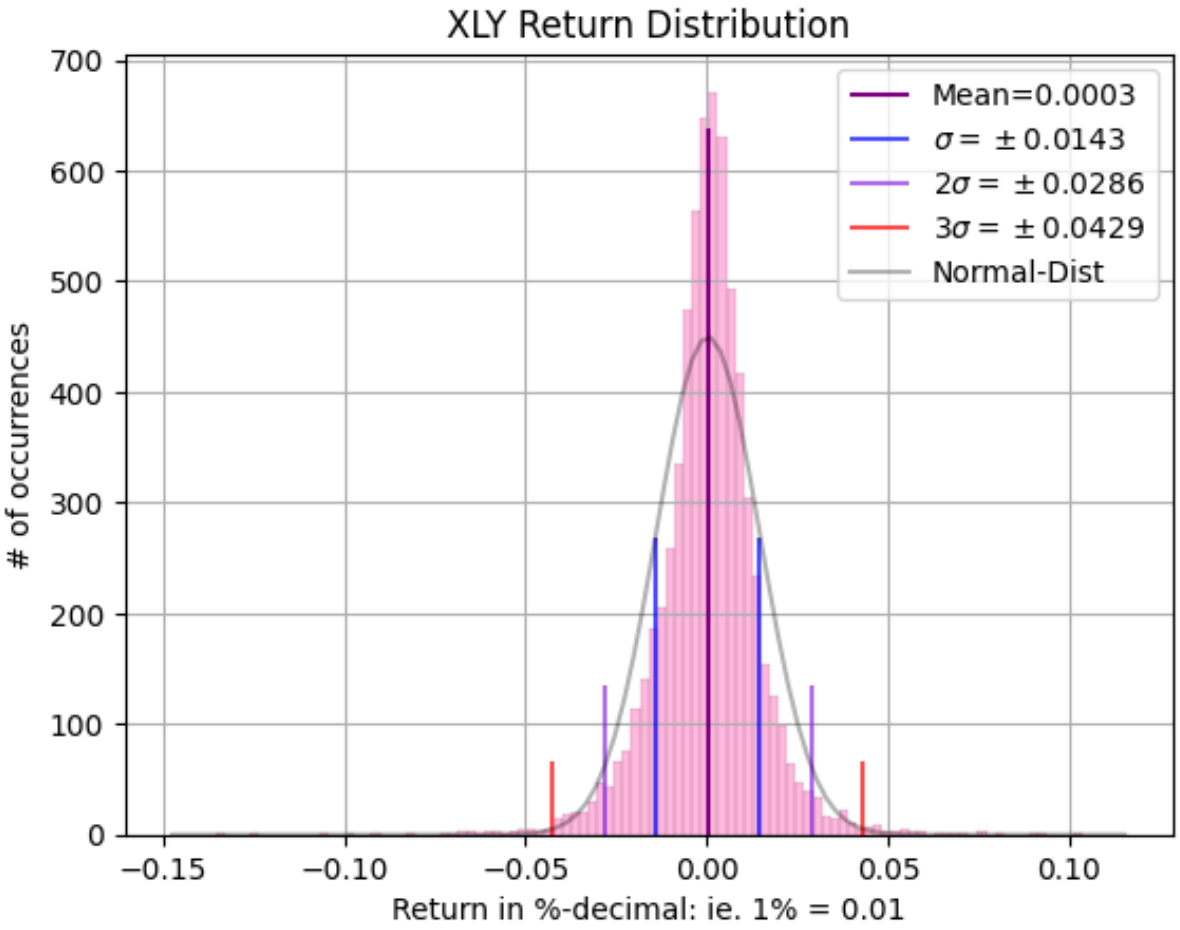

XLK Return Distribution
Mean=0.0003
σ = ±0.0163
2σ = ±0.0326
3σ = ±0.0489
Normal-Dist
# of occurrences
0
100
200
300
400
500
600
700
800
−0.15
−0.10
−0.05
0.00
0.05
0.10
0.15
Return in %-decimal: ie. 1% = 0.01


XLP Return Distribution
Mean=0.0003
σ = ±0.0094
2σ = ±0.0189
3σ = ±0.0283
Normal-Dist
# of occurrences
0
100
200
300
400
500
600
700
−0.100
−0.075
−0.050
−0.025
0.000
0.025
0.050
0.075
0.100
Return in %-decimal: ie. 1% = 0.01


XLRE Return Distribution
Mean=0.0003
$\sigma = \pm 0.0128$
$2\sigma = \pm 0.0255$
$3\sigma = \pm 0.0383$
Normal-Dist
# of occurrences
0
50
100
150
200
250
300
−0.20
−0.15
−0.10
−0.05
0.00
0.05
0.10
Return in %-decimal: ie. 1% = 0.01


XLU Return Distribution
Mean=0.0003
σ = ±0.0121
2σ = ±0.0242
3σ = ±0.0363
Normal-Dist
# of occurrences
0
100
200
300
400
500
600
700
−0.10
−0.05
0.00
0.05
0.10
Return in %-decimal: ie. 1% = 0.01


XLV Return Distribution
Mean=0.0003
σ = ±0.0112
2σ = ±0.0224
3σ = ±0.0336
Normal-Dist
# of occurrences
0
100
200
300
400
500
600
700
−0.10
−0.05
0.00
0.05
0.10
Return in %-decimal: ie. 1% = 0.01




# Task 2

# Annual Volatility

Since log-returns are additive for compounding. We just need to group by year and sum for annualized returns across our time period.

Volatility I will elect to take rolling 90-day windows daily, then compute the average annually, then scale up to annualized by multiplying by $\sqrt{252}.$ Because volatility scales by $\sqrt{\text{time}}$.

In [28]:
```
df_vol = log_returns.groupby(log_returns.index.year).std() * np.sqrt(2
df_vol
```

Out[28]:

| Ticker | XLB | XLC | XLE | XLF | XLI | XLK | XLP |
|---|---|---|---|---|---|---|---|
| Date | | | | | | | |
| 2000 | 0.309386 | NaN | 0.285607 | 0.331983 | 0.245537 | 0.434862 | 0.250427 |
| 2001 | 0.270129 | NaN | 0.258497 | 0.252794 | 0.269053 | 0.469465 | 0.188387 |
| 2002 | 0.288892 | NaN | 0.288284 | 0.311292 | 0.282543 | 0.417713 | 0.194693 |
| 2003 | 0.187335 | NaN | 0.159788 | 0.197232 | 0.176811 | 0.251905 | 0.135605 |
| 2004 | 0.170972 | NaN | 0.178713 | 0.122632 | 0.127967 | 0.173093 | 0.098850 |
| 2005 | 0.159742 | NaN | 0.256584 | 0.118993 | 0.117858 | 0.121619 | 0.090383 |
| 2006 | 0.178026 | NaN | 0.249429 | 0.116614 | 0.120169 | 0.141538 | 0.087004 |
| 2007 | 0.225392 | NaN | 0.250980 | 0.240719 | 0.160724 | 0.163587 | 0.108563 |
| 2008 | 0.471753 | NaN | 0.614376 | 0.715477 | 0.384242 | 0.408696 | 0.241994 |
| 2009 | 0.337755 | NaN | 0.352526 | 0.642079 | 0.327775 | 0.257241 | 0.172573 |
| 2010 | 0.249590 | NaN | 0.234202 | 0.254628 | 0.220508 | 0.187157 | 0.121173 |
| 2011 | 0.310824 | NaN | 0.318000 | 0.334177 | 0.274064 | 0.226509 | 0.148335 |
| 2012 | 0.176333 | NaN | 0.182085 | 0.175048 | 0.153067 | 0.147316 | 0.093845 |
| 2013 | 0.143683 | NaN | 0.143047 | 0.143168 | 0.129368 | 0.115308 | 0.113382 |
| 2014 | 0.141166 | NaN | 0.184764 | 0.128607 | 0.137210 | 0.126536 | 0.096439 |
| 2015 | 0.184930 | NaN | 0.248918 | 0.176189 | 0.160834 | 0.177486 | 0.138115 |
| 2016 | 0.178331 | NaN | 0.245992 | 0.187039 | 0.146979 | 0.153373 | 0.118053 |
| 2017 | 0.104121 | NaN | 0.132872 | 0.130699 | 0.091853 | 0.103987 | 0.076377 |
| 2018 | 0.191717 | 0.223323 | 0.222950 | 0.196180 | 0.190987 | 0.233688 | 0.145219 |
| 2019 | 0.156700 | 0.164203 | 0.189900 | 0.155148 | 0.155862 | 0.180847 | 0.107287 |
| 2020 | 0.385618 | 0.334391 | 0.611573 | 0.455230 | 0.397672 | 0.403767 | 0.278610 |
| 2021 | 0.170327 | 0.162863 | 0.295374 | 0.188600 | 0.156613 | 0.192899 | 0.111013 |
| 2022 | 0.252493 | 0.309286 | 0.354034 | 0.244616 | 0.218981 | 0.327994 | 0.176506 |
| 2023 | 0.168363 | 0.195009 | 0.229192 | 0.162461 | 0.147544 | 0.184957 | 0.113827 |
| 2024 | 0.136177 | 0.154466 | 0.179006 | 0.140974 | 0.136568 | 0.222421 | 0.098182 |
| 2025 | 0.199776 | 0.184982 | 0.248423 | 0.191217 | 0.187933 | 0.274180 | 0.138412 |
| 2026 | 0.183233 | 0.171835 | 0.230520 | 0.149908 | 0.180457 | 0.277967 | 0.159132 |

In [29]:
```
df_yearly = df['Close'].groupby(df.index.year)
```

In [30]:
```
yearly_returns = {}

for year in df_yearly:
    yr = year[0]
    df1 = year[1]


    lg_rtn = np.log(df1) - np.log(df1).shift(1)

    yearly_returns.update({yr: lg_rtn.sum()})

yearly_return_summary = pd.DataFrame(yearly_returns).T
yearly_return_summary = np.exp(yearly_return_summary) - 1
```

# Annual Return Summary

Below is a table of annual returns.

## Notable Years:

- 2000 was the Dot-Com era crash that led to large negative returns in technology companies like Apple.
- 2008 was the Great Financial Crisis, so we see stocks with a large negative return that year
- 2020, although Intra-Year 2020 saw a massive crash in Q1, the stock market recovered and rallied as a whole throughout the year as liquidity was pumped in the system and new market participants took discretionary money on stock trading, (ie. Robinhood stock-pickers).
- 2022 we can see the 'Mini Bear Market' of interest rate hiking following COVID

Calculation:

To take log-returns back into percentage returns, we need to exponentiate them.

$$r_{\text{Compounded}} = e^{r_{\text{log}} } - 1$$

In [31]:
```
yearly_return_summary
```

Out[31]:

| Ticker | XLB | XLC | XLE | XLF | XLI | XLK | |
|---|---|---|---|---|---|---|---|
| **2000** | -0.142583 | 0.000000 | 0.268888 | 0.307566 | 0.095412 | -0.435175 | 0.2 |
| **2001** | 0.041926 | 0.000000 | -0.183253 | -0.068147 | -0.063743 | -0.186440 | -0.0 |
| **2002** | -0.060304 | 0.000000 | -0.140377 | -0.146387 | -0.245730 | -0.396750 | -0.1 |
| **2003** | 0.339032 | 0.000000 | 0.232182 | 0.259131 | 0.285487 | 0.315640 | 0.0 |
| **2004** | 0.140480 | 0.000000 | 0.339839 | 0.110175 | 0.178394 | 0.051218 | 0.0 |
| **2005** | 0.051411 | 0.000000 | 0.455487 | 0.065884 | 0.036494 | 0.005408 | 0.0 |
| **2006** | 0.159147 | 0.000000 | 0.127144 | 0.169318 | 0.127622 | 0.099993 | 0.1 |
| **2007** | 0.228969 | 0.000000 | 0.418480 | -0.195698 | 0.127845 | 0.151106 | 0. |
| **2008** | -0.435069 | 0.000000 | -0.390705 | -0.539958 | -0.376934 | -0.402979 | -0.1 |
| **2009** | 0.429407 | 0.000000 | 0.159852 | 0.162653 | 0.177088 | 0.458361 | 0.1 |
| **2010** | 0.169050 | 0.000000 | 0.180497 | 0.096949 | 0.254285 | 0.097669 | 0.1 |
| **2011** | -0.116350 | 0.000000 | 0.020764 | -0.189217 | -0.018037 | 0.014858 | 0. |
| **2012** | 0.115954 | 0.000000 | 0.023694 | 0.251425 | 0.124586 | 0.136882 | 0.1 |
| **2013** | 0.229103 | 0.000000 | 0.235139 | 0.316712 | 0.372531 | 0.221757 | 0.2 |
| **2014** | 0.080480 | 0.000000 | -0.073594 | 0.156887 | 0.119293 | 0.190079 | 0. |
| **2015** | -0.088247 | 0.000000 | -0.218634 | -0.017701 | -0.042048 | 0.056737 | 0.0 |
| **2016** | 0.186546 | 0.000000 | 0.280678 | 0.248247 | 0.216346 | 0.165355 | 0.0 |
| **2017** | 0.232933 | 0.000000 | -0.019934 | 0.206506 | 0.232463 | 0.330732 | 0.1 |
| **2018** | -0.160699 | -0.168805 | -0.195426 | -0.130832 | -0.137602 | -0.028770 | -0.0 |
| **2019** | 0.234935 | 0.293248 | 0.095823 | 0.307759 | 0.284266 | 0.497592 | 0. |
| **2020** | 0.218895 | 0.254309 | -0.332715 | -0.026848 | 0.088475 | 0.409737 | 0.1 |
| **2021** | 0.286644 | 0.177063 | 0.530376 | 0.366520 | 0.240806 | 0.369775 | 0.1 |
| **2022** | -0.111397 | -0.380601 | 0.593854 | -0.116722 | -0.047858 | -0.284057 | -0.0 |
| **2023** | 0.125718 | 0.508717 | 0.029843 | 0.116392 | 0.178772 | 0.575120 | -0.0 |
| **2024** | 0.003252 | 0.354711 | 0.044381 | 0.300030 | 0.185053 | 0.249011 | 0.1 |
| **2025** | 0.111721 | 0.224838 | 0.066315 | 0.151884 | 0.197651 | 0.249014 | 0.0 |
| **2026** | 0.126820 | -0.089206 | 0.548828 | 0.030106 | 0.050345 | 0.274365 | 0. |

# Task 3

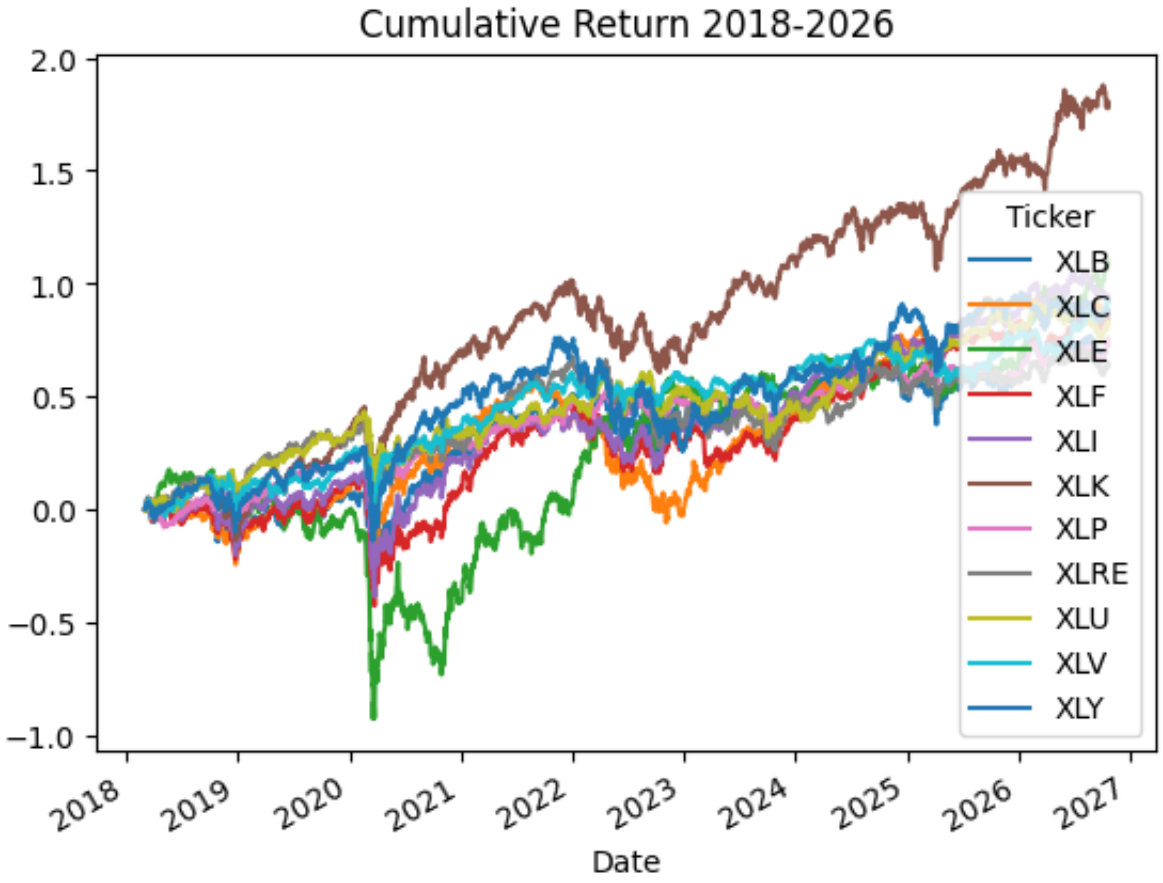
Back to Top

# Data Visualization

# Data Visualization

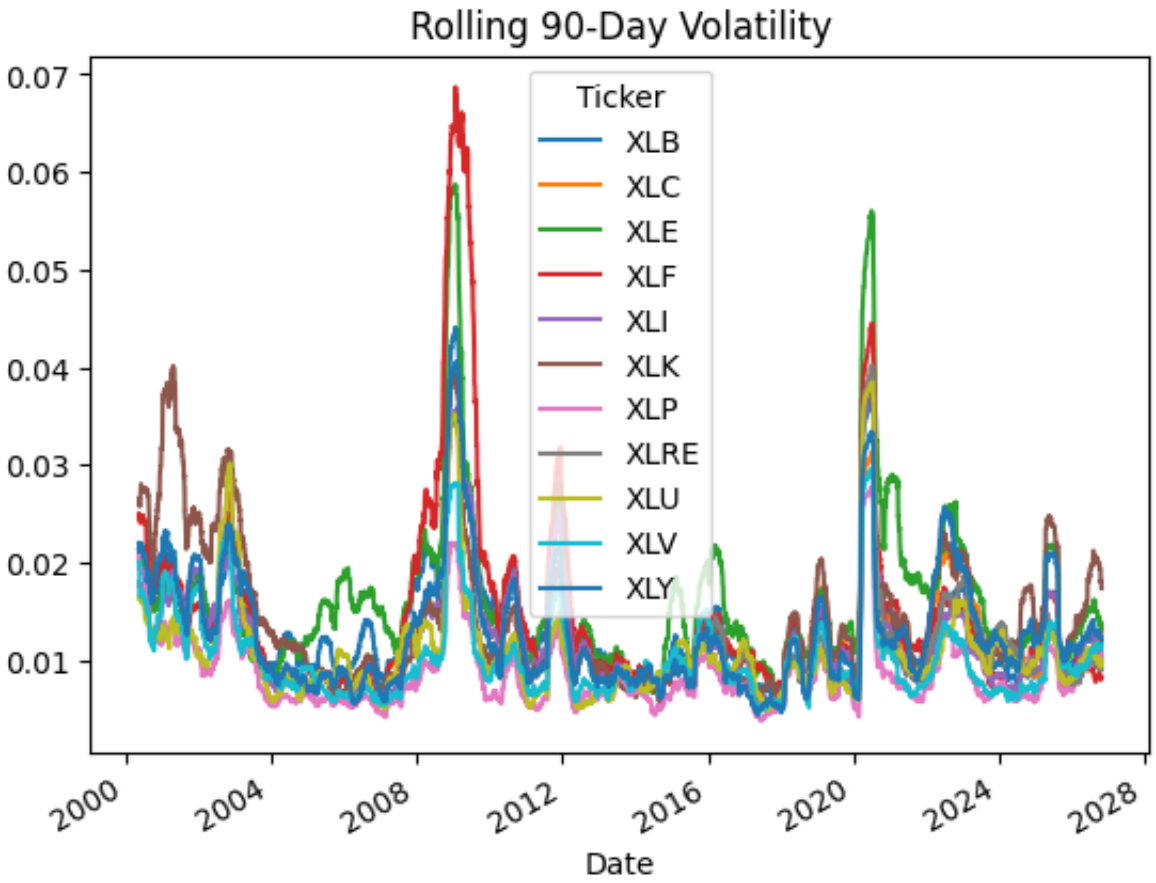
In [32]:
```
log_returns[log_returns.index > '03-01-2018'].cumsum().plot(title='Cum
```

Out[32]:
```
<Axes: title={'center': 'Cumulative Return 2018-2026'}, xlabel='Date'
>
```




In [33]:
```
log_returns.rolling(window=90).std().plot(title='Rolling 90-Day Volati
```

Out[33]:
```
<Axes: title={'center': 'Rolling 90-Day Volatility'}, xlabel='Date'>
```




## Correlation Matrix

In [35]:
```
log_returns.corr()
```

Out[35]:

| Ticker | XLB | XLC | XLE | XLF | XLI | XLK | XLP |
|---|---|---|---|---|---|---|---|
| Ticker | | | | | | | |
| **XLB** | 1.000000 | 0.646519 | 0.675911 | 0.706546 | 0.824636 | 0.614886 | 0.583809 |
| **XLC** | 0.646519 | 1.000000 | 0.412002 | 0.663857 | 0.666752 | 0.776353 | 0.522465 |
| **XLE** | 0.675911 | 0.412002 | 1.000000 | 0.582137 | 0.638567 | 0.446655 | 0.448478 |
| **XLF** | 0.706546 | 0.663857 | 0.582137 | 1.000000 | 0.787813 | 0.624240 | 0.572597 |
| **XLI** | 0.824636 | 0.666752 | 0.638567 | 0.787813 | 1.000000 | 0.736374 | 0.623750 |
| **XLK** | 0.614886 | 0.776353 | 0.446655 | 0.624240 | 0.736374 | 1.000000 | 0.441295 |
| **XLP** | 0.583809 | 0.522465 | 0.448478 | 0.572597 | 0.623750 | 0.441295 | 1.000000 |
| **XLRE** | 0.647176 | 0.576871 | 0.424604 | 0.624653 | 0.659498 | 0.527758 | 0.671172 |
| **XLU** | 0.521762 | 0.410007 | 0.475293 | 0.494032 | 0.552083 | 0.431792 | 0.602085 |
| **XLV** | 0.630207 | 0.610142 | 0.473182 | 0.621854 | 0.704082 | 0.626628 | 0.614156 |
| **XLY** | 0.721803 | 0.804859 | 0.508681 | 0.739198 | 0.802626 | 0.735980 | 0.588977 |

# Task 4

Back To Top

## Written Assesment

Presented here is a dynamic and interactive data analysis document. The user can build a portfolio of tickers to perfrom analysis and run the coding cells producing analysis. It all works sequentially, so one can perform another analysis by just going back to the top and choosing a different date range and set of tickers.

Comparing 2-3 stocks, intially I chose Apple, Lockheed Martin, and TORM ( A shipping tanker company). These different tickers exhibit different return and volatility characteristics. You can see that Lockheed Martin exhibits the lowest volatility, as a large stable company in the defense sector, this makes sense. Compared to Apple in the technology sector, we can see Apple's higher-cumulative return matches their higher volatility.

From a macroeconomic perspective you can see the impact of large exogenous events on all of these stocks. Most notably 2000 era Dot-Com crash in Apple.

Volatility spikes and negative returns in 2007,2008 due to the Great Financial Crisis. You can also note 2020 COVID-era crash and subsequent rally, followed by 2022 interest-rate hiking adding volatility. More recently you can see 2025 "Liberation Day" volatility spikes from President Trump's aggressive tarrif policy announcements. As an tanker company that deals in oil-markets, you can see Torm's stock rally during 2022 Russian oil embargo, and sinking of oil vessels in Red Sea and 2026 war on Iran. TRMD's exposure to these regional crises has been limited, while their company has benefited from the higher-price point demanded in international markets for shipping tankers.

In [ ]: